

Lab Manual

Object Oriented Programming with C++

CSE2001

Submitted by

Name:- **Vijay Rajesh R**

Reg. No:- **23BAI10917**

Bachelor of Technology

In

CSE (AIML)



**Submitted to School of Computing Science and
Engineering**

VIT Bhopal University

Date of Submission : 18.12.2023

Course Code CSE2001	Object Oriented Programming with C++	Course Type LTP	Credits 4
------------------------	--------------------------------------	--------------------	--------------

Indicative list of Experiments

No.	Description of Experiments	PAGE
1	Design a class to represent a bank account. Include the following members. Data Members • Name of the depositor • Account number • Type of account • Balance amount in the account Methods • To assign initial values • To deposit an amount • To withdraw an amount after checking balance • To display the name and balance Incorporate a constructor to provide initial values.	5-7
2	Guess-the-number-game: Write a program that plays the game of “guess the number” as follows: Your program chooses the number to be guessed by selecting an integer at random in the range 1 to 1000. The program then displays the following: I have a number between 1 and 1000. Can you guess my number? Please type your first guess. The player then types a first guess. The program responds with one of the following: 1. Excellent ! you guessed the number! Would like to play again (y or n)? 2. Too low. Try again. 3. Too high. Try again.	8-9

	If the payer's guess is incorrect, your program should loop until the player finally get the number right. Your program should keep telling the player Too high or Too low to help the player "zero in" on the correct answer.	
3	type of account. From this device the classes Curr-acct and Sav-acct to make them more specific to their requirements. Include the necessary methods in order to achieve the following tasks. • Accept deposit from a customer and update the balance • Display the balance • Compute and deposit interest • Permit withdrawal and update the balance • Check for the minimum balance, impose penalty, if necessary and update the balance. Do not use any constructors. Use methods to initialize the class members. type of account. From this device the classes Curr-acct and Sav-acct to make them more specific to their requirements. Include the necessary methods in order to achieve the following tasks. • Accept deposit from a customer and update the balance • Display the balance • Compute and deposit interest • Permit withdrawal and update the balance • Check for the minimum balance, impose penalty, if necessary and update the balance. Do not use any constructors. Use methods to initialize the class members	10-16
4	An election is contested by 5 candidates. The candidates are numbered 1 to 5 and the voting is done by marking the candidate number on the ballot paper. Write a C++ program to read the ballots and count the votes cast for each candidate using an array. In case, a number read is outside the range 1 to 5, the ballot should be considered as a 'spoilt ballot' and the program should also count the number of spoilt ballots.	17-18
5	Develop a program which will read a string and rewrite it in the alphabetical order. For example, the word STRING should be written as GINRST	19
6	Create a class by name date with the member data day, month and year. Perform the following: • Overload all relational operators <, <=, >, >=, ==, !=	20-23

	• Overload ++ operator to increment a date by one day • Overload + to add given number of days to find the next date Provide the necessary function to use the statement like days=dt; where days is an int variable and dt is an object of date class. The statement is intended to assign the number of days elapsed in the current year of the date to the variable days. Note that this is a case of conversion from derived type to basic type.	
7	Develop a program to sort a file consisting of books' details in the alphabetical order of author names. • The details of books include book_id, author_name, price, no_of_pages, publisher, year_of_publishing	24-25
8	Design a class template by name Vector and perform the following: • Find the smallest of the element in the Vector • Search for an element in the Vector Find the average of the element in the array	26-29
9	• Design a generic function for finding the largest of three numbers.	30-31

EXPERIMENT 1

```
#include<iostream>
using namespace std;
class person
{
    char name[20];
    int account;
    char type[15];
    int balance,c;
    //int dsp;
    //int reamount;

public:
    void getdata(void);
    void display(void);
    void calculate(void);
    void disp(void);
};

void person :: getdata(void)
{
    int a;
    //char type[15];
    cout<<"\n Enter account holder name ";
    cin>>name;
    cout<<"\n Enter account number ";
    cin>>account;
    cout<<"\n Press 1 for saving account, press 2 for current
account ";
    cin>>a;
```

```

if(a==1){
    cout<<" Saving Account";
    }
else if(a==2)
{

    cout<<" Current Account";
    }
else{
    cout<<"Invalid Account Type";
}

cout<<"\n Enter Amount to be deposited ";
cin>>balance;

}

void person :: display(void)
{
cout<<"\nName: "<<name;
cout<<"\nAccount "<<account;
cout<<"\nAccount Type "<<type;
cout<<"\nbalance "<<balance;
cout<<endl;
}

void person :: calculate(void)
{
int a,b,e;
cout<<" press 1 to withdraw, 2 to deposit ";
cin>>a;
if(a==1)
{
    cout<<" Enter the amount you want to withdraw ";
    cin>>b;
    c = balance - b;
}
else if(a==2)
{

```

```

cout<<"  Enter the amount you want to deposit ";
cin>>b;
c = balance + b;

}
}
void person :: disp(void)
{
cout<<"\n Name: "<<name;
cout<<"\n Account "<<account;
cout<<"\n Account type "<<type;
cout<<"\n Current balance "<<c;
cout<<endl;
}
int main()
{
person a;
a.getdata();
a.display();
a.calculate();
a.disp();
}

```

OUTPUT

Enter account holder name Harsh

Enter account number 10010904

Press 1 for saving account, press 2 for current account 1

Saving Account

Enter Amount to be deposited 5000

Name: Nandini

Account 1001089

Account Type T a

balance 5000

press 1 to withdraw, 2 to deposit 2

Enter the amount you want to deposit 600

Name: Nandini

Account 1001089

Account type Saving

Current balance 5600

EXPERIMENT 2

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
int main()
{
    int x, y, a = 0;
    srand(time(0));

    x = rand() % 100 + 1;
    cout<<"\n\n          WELCOME TO THE GAME!!!\n\n";
    cout << "  CHOOSE A RANDOM NUMBER BETWEEN 1 TO
100  \n\n";
    do
    {
        cout << "Enter Your Guess :- ";
        cin >> y;
        a++;
        if (y > x)
            cout << "Too high!\n\n";
        else if (y < x)
            cout << "Too low!\n\n";
        else
            cout << "\nCorrect! YOU ARE THE WINNER\n" << a
<< " th guess";
    } while (y != x);
```

```
return 0; }
```

OUTPUT

WELCOME TO THE GAME!!!

CHOOSE A RANDOM NUMBER BETWEEN 1 TO 100

Enter Your Guess :- 50

Too low!

Enter Your Guess :- 80

Too high!

Enter Your Guess :- 70

Too high!

Enter Your Guess :- 65

Too high!

Enter Your Guess :- 60

Too high!

Enter Your Guess :- 55

Correct! YOU ARE THE WINNER

6 th guess

EXPERIMENT 3

```
#include <iostream>
#include <string>

class Account {
public:
    std::string customerName;
    std::string accountNumber;
    double balance;

public:
    void initializeAccount() {
        std::cout << "Enter customer name: ";
        std::cin >> customerName;
        std::cout << "Enter account number: ";
        std::cin >> accountNumber;
        balance = 0.0;
    }

    void acceptDeposit(double amount) {
        balance += amount;
        std::cout << "Deposit of $" << amount << "
accepted. New balance: $" << balance << std::endl;
    }

    void displayBalance() {
        std::cout << "Account Number: " << accountNumber
<< std::endl;
        std::cout << "Customer Name: " << customerName <<
std::endl;
        std::cout << "Balance: $" << balance << std::endl;
    }

    virtual void computeInterest() {}
}
```

```

    virtual void withdraw(double amount ) {}

    virtual void checkMinimumBalance() {}

    virtual ~Account() {}
};

class SavingAccount : public Account {
private:
    double interestRate;

public:
    SavingAccount() {
        interestRate = 5.0; // Default interest rate for
savings account
    }

    void computeInterest() {
        double interest = balance * (interestRate / 100);
        acceptDeposit(interest);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            std::cout << "Withdrawal of $" << amount << "
successful. New balance: $" << balance << std::endl;
        } else {
            std::cout << "Insufficient balance for
withdrawal." << std::endl;
        }
    }

    void checkMinimumBalance() {
        if (balance < 1000) {
            balance -= 50; // Penalty for falling below
minimum balance
            std::cout << "Minimum balance penalty of $50
imposed. New balance: $" << balance << std::endl;

```

```

    }
}
};

class CurrentAccount : public Account {
public:
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            std::cout << "Withdrawal of $" << amount << "
successful. New balance: $" << balance << std::endl;
        } else {
            std::cout << "Insufficient balance for
withdrawal." << std::endl;
        }
    }

    void checkMinimumBalance() {
        if (balance < 1000) {
            balance -= 100; // Penalty for falling below
minimum balance
            std::cout << "Minimum balance penalty of $100
imposed. New balance: $" << balance << std::endl;
        }
    }
};

int main() {
    int choice;
    Account* acc = nullptr;

    do {
        std::cout << "1. Create a new Saving Account" <<
std::endl;
        std::cout << "2. Create a new Current Account" <<
std::endl;
        std::cout << "3. Deposit" << std::endl;
        std::cout << "4. Withdraw" << std::endl;
    }
}

```

```

        std::cout << "5. Compute Interest (Savings
Account)" << std::endl;
        std::cout << "6. Check Minimum Balance" <<
std::endl;
        std::cout << "7. Display Balance" << std::endl;
        std::cout << "8. Exit" << std::endl;
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                if (acc != nullptr) {
                    delete acc;
                }
                acc = new SavingAccount();
                acc->initializeAccount();
                break;
            case 2:
                if (acc != nullptr) {
                    delete acc;
                }
                acc = new CurrentAccount();
                acc->initializeAccount();
                break;
            case 3:
                if (acc == nullptr) {
                    std::cout << "Create an account
first." << std::endl;
                } else {
                    double amount;
                    std::cout << "Enter the deposit
amount: $";

                    std::cin >> amount;
                    acc->acceptDeposit(amount);
                }
                break;
            case 4:
                if (acc == nullptr) {

```

```

        std::cout << "Create an account
first." << std::endl;
    } else {
        double amount;
        std::cout << "Enter the withdrawal
amount: $";

        std::cin >> amount;
        acc->withdraw(amount);
    }
    break;
case 5:
    if (dynamic_cast<SavingAccount*>(acc) !=
nullptr) {
        SavingAccount* savingAcc =
dynamic_cast<SavingAccount*>(acc);
        savingAcc->computeInterest();
    } else {
        std::cout << "Interest can only be
computed for Saving Accounts." << std::endl;
    }
    break;
case 6:
    if (acc == nullptr) {
        std::cout << "Create an account
first." << std::endl;
    } else {
        acc->checkMinimumBalance();
    }
    break;
case 7:
    if (acc == nullptr) {
        std::cout << "Create an account
first." << std::endl;
    } else {
        acc->displayBalance();
    }
    break;
case 8:
    if (acc != nullptr) {

```

```
        delete acc;
    }
    std::cout << "Exiting program. Goodbye!"
<< std::endl;
    break;
    default:
        std::cout << "Invalid choice. Please try
again." << std::endl;
        break;
    }
} while (choice != 8);

return 0;
}
```

OUTPUT

1. Create a new Saving Account
2. Create a new Current Account
3. Deposit
4. Withdraw
5. Compute Interest (Savings Account)
6. Check Minimum Balance
7. Display Balance
8. Exit

Enter your choice: 1

Enter customer name: Nandini

Enter account number: 1001567

1. Create a new Saving Account
2. Create a new Current Account
3. Deposit
4. Withdraw
5. Compute Interest (Savings Account)
6. Check Minimum Balance
7. Display Balance
8. Exit

Enter your choice: 3

Enter the deposit amount: \$6000

Deposit of \$6000 accepted. New balance: \$6000

1. Create a new Saving Account
2. Create a new Current Account
3. Deposit
4. Withdraw
5. Compute Interest (Savings Account)
6. Check Minimum Balance
7. Display Balance
8. Exit

Enter your choice: 4

Enter the withdrawal amount: \$500

Withdrawal of \$500 successful. New balance: \$4500

1. Create a new Saving Account
2. Create a new Current Account
3. Deposit

4. Withdraw
5. Compute Interest (Savings Account)
6. Check Minimum Balance
7. Display Balance
8. Exit

Enter your choice: 7

Account Number: 1001567

Customer Name: Nandini

Balance: \$4500

EXPERIMENT 4

```
#include <iostream>

int main() {
    const int numCandidates = 5;
    int candidateVotes[numCandidates] = {0}; // Initialize
votes for each candidate to zero
    int spoiledBallots = 0;
    int ballot;

    std::cout << "Enter the candidate number on each
ballot (1 to 5) or any number outside this range to
indicate a spoiled ballot." << std::endl;
    std::cout << "Enter 0 to end input." << std::endl;

    while (true) {
        std::cout << "Enter a ballot (0 to end): ";
        std::cin >> ballot;

        if (ballot == 0) {
            break;
        }
    }
}
```

```

        if (ballot >= 1 && ballot <= numCandidates) {
            candidateVotes[ballot - 1]++;
        } else {
            spoiltBallots++;
        }
    }

    std::cout << "Vote count for each candidate:" <<
std::endl;
    for (int i = 0; i < numCandidates; i++) {
        std::cout << "Candidate " << (i + 1) << ": " <<
candidateVotes[i] << " votes" << std::endl;
    }

    std::cout << "Spoilt Ballots: " << spoiltBallots <<
std::endl;
    return 0;
}

```

OUTPUT

Enter the candidate number on each ballot (1 to 5) or any number outside this range to indicate a spoilt ballot.

Enter 0 to end input.

Enter a ballot (0 to end): 1

Enter a ballot (0 to end): 2

Enter a ballot (0 to end): 5

Enter a ballot (0 to end): 5

Enter a ballot (0 to end): 4

Enter a ballot (0 to end): 5

Enter a ballot (0 to end): 3

Enter a ballot (0 to end): 0

Vote count for each candidate:

Candidate 1: 1 votes

Candidate 2: 1 votes
Candidate 3: 1 votes
Candidate 4: 1 votes
Candidate 5: 3 votes
Spoilt Ballots: 0

EXPERIMENT 5

```
#include <iostream>
#include <algorithm>

int main() {
    // Get input string
    std::cout << "Enter a string: ";
    std::string inputString;
    std::getline(std::cin, inputString);

    // Sort the string in alphabetical order
    std::sort(inputString.begin(), inputString.end());

    // Display the result
    std::cout << "String in alphabetical order: " <<
inputString << std::endl;

    return 0;
}
```

```
}
```

OUTPUT

Enter a string: hello

String in alphabetical order: ehlllo

EXPERIMENT 6

```
#include <iostream>

class Date {
private:
    int day, month, year;

public:
    // Constructor
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {}

    // Overload relational operators
    bool operator<(const Date& other) const;
    bool operator<=(const Date& other) const;
    bool operator>(const Date& other) const;
    bool operator>=(const Date& other) const;
    bool operator==(const Date& other) const;
    bool operator!=(const Date& other) const;

    // Overload ++ operator to increment the date by one
    day
    Date& operator++(); // Prefix increment

    // Overload + operator to add given number of days
    Date operator+(int days) const;

    // Conversion operator
```

```

    operator int() const;
    // Display the date
    void display() const;
};

// Function definitions

bool Date::operator<(const Date& other) const {
    if (year != other.year)
        return year < other.year;
    if (month != other.month)
        return month < other.month;
    return day < other.day;
}

bool Date::operator<=(const Date& other) const {
    return (*this < other) || (*this == other);
}

bool Date::operator>(const Date& other) const {
    return !(*this <= other);
}

bool Date::operator>=(const Date& other) const {
    return !(*this < other);
}

bool Date::operator==(const Date& other) const {
    return (day == other.day) && (month == other.month) &&
(year == other.year);
}

bool Date::operator!=(const Date& other) const {
    return !(*this == other);
}

Date& Date::operator++() {
    // Increment the date by one day
    if (++day > 31) {

```

```

        day = 1;
        if (++month > 12) {
            month = 1;
            ++year;
        }
    }
    return *this;
}

Date Date::operator+(int days) const {
    // Add given number of days to the date
    Date result = *this;
    for (int i = 0; i < days; ++i) {
        ++result;
    }
    return result;
}

Date::operator int() const {
    // Conversion to int (number of days elapsed in the
    current year)
    int days = day;
    for (int m = 1; m < month; ++m) {
        switch (m) {
            case 4: case 6: case 9: case 11:
                days += 30;
                break;
            case 2:
                days += (year % 4 == 0 && (year % 100 != 0
|| year % 400 == 0)) ? 29 : 28;
                break;
            default:
                days += 31;
        }
    }
    return days;
}

void Date::display() const {

```

```

        std::cout << day << '/' << month << '/' << year;
    }
    int main() {
        // Example usage
        Date dt1(15, 11, 2023);
        Date dt2(20, 11, 2023);

        std::cout << "Comparison: " << (dt1 < dt2) <<
std::endl;

        std::cout << "Increment by one day: ";
        ++dt1;
        dt1.display();
        std::cout << std::endl;

        int days = static_cast<int>(dt1);
        std::cout << "Days elapsed in the current year: " <<
days << std::endl;

        int additionalDays = 5;
        Date dt3 = dt1 + additionalDays;
        std::cout << "Adding " << additionalDays << " days: ";
        dt3.display();
        std::cout << std::endl;

        return 0;
    }

```

OUTPUT

Comparison: 1

Increment by one day: 16/11/2023

Days elapsed in the current year: 320

Adding 5 days: 21/11/2023

EXPERIMENT 7

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

// Structure to represent book details
struct Book {
    int book_id;
    std::string author_name;
    double price;
    int no_of_pages;
    std::string publisher;
    int year_of_publishing;
};

std::vector<Book> readBooksFromFile(const std::string&
filename) {
    std::vector<Book> books;
    std::ifstream file(filename);

    if (file.is_open()) {
        Book book;
        while (file >> book.book_id >> book.author_name >>
book.price >> book.no_of_pages >> book.publisher >>
book.year_of_publishing) {
            books.push_back(book);
        }

        file.close();
    } else {
        std::cerr << "Error opening file: " << filename <<
std::endl;
    }

    return books;
}

// Function to write book details to a file
```

```

void writeBooksToFile(const std::string& filename, const
std::vector<Book>& books) {
    std::ofstream file(filename);

    if (file.is_open()) {
        for (const auto& book : books) {
            file << book.book_id << ' ' <<
book.author_name << ' ' << book.price << ' ' <<
book.no_of_pages << ' ' << book.publisher << ' ' <<
book.year_of_publishing << '\n';
        }

        file.close();
    } else {
        std::cerr << "Error opening file: " << filename <<
std::endl;
    }
}

// Comparator function to sort books based on author names
bool compareBooks(const Book& book1, const Book& book2) {
    return book1.author_name < book2.author_name;
}

int main() {
    // Specify input and output file names
    std::string inputFileName = "books_input.txt";
    std::string outputFileName =
"books_sorted_output.txt";

    // Read book details from the input file
    std::vector<Book> books =
readBooksFromFile(inputFileName);

    // Sort the books based on author names
    std::sort(books.begin(), books.end(), compareBooks);

    // Write the sorted books to the output file
    writeBooksToFile(outputFileName, books);
}

```

```

        std::cout << "Books sorted and written to: " <<
outputFileName << std::endl;
        return 0;
}

```

EXPERIMENT 8

```

#include <iostream>
#include <vector>

template <typename T>
class Vector {
private:
    std::vector<T> elements;

public:
    // Constructor to initialize the vector with elements
    Vector(const std::vector<T>& inputElements) :
elements(inputElements) {}

    // Function to find the smallest element in the vector
    T findSmallest() const;

    // Function to search for an element in the vector
    bool searchElement(const T& target) const;

    // Function to find the average of the elements in the
vector
    double findAverage() const;
};

template <typename T>
T Vector<T>::findSmallest() const {
    if (elements.empty()) {
        // Handle the case when the vector is empty
        throw std::runtime_error("Vector is empty");
    }
}

```

```

    T smallest = elements[0];
    for (const T& element : elements) {
        if (element < smallest) {
            smallest = element;
        }
    }

    return smallest;
}

template <typename T>
bool Vector<T>::searchElement(const T& target) const {
    for (const T& element : elements) {
        if (element == target) {
            return true; // Element found
        }
    }

    return false; // Element not found
}

template <typename T>
double Vector<T>::findAverage() const {
    if (elements.empty()) {
        // Handle the case when the vector is empty
        throw std::runtime_error("Vector is empty");
    }

    T sum = 0;
    for (const T& element : elements) {
        sum += element;
    }

    return static_cast<double>(sum) / elements.size();
}

int main() {
    // Example usage with integers
    std::vector<int> intElements = {5, 2, 8, 1, 7};

```

```

    Vector<int> intVector(intElements);

    try {
        int smallestInt = intVector.findSmallest();
        std::cout << "Smallest element in the vector: " <<
smallestInt << std::endl;

        int searchTarget = 8;
        bool found =
intVector.searchElement(searchTarget);
        std::cout << "Search for element " << searchTarget
<< ": " << (found ? "Found" : "Not found") << std::endl;

        double averageInt = intVector.findAverage();
        std::cout << "Average of elements in the vector: "
<< averageInt << std::endl;
    } catch (const std::runtime_error& e) {
        std::cerr << "Error: " << e.what() << std::endl;
    }

    // Example usage with doubles
    std::vector<double> doubleElements = {3.5, 1.2, 4.8,
2.1, 5.7};
    Vector<double> doubleVector(doubleElements);

    try {
        double smallestDouble =
doubleVector.findSmallest();
        std::cout << "Smallest element in the vector: " <<
smallestDouble << std::endl;

        double searchTargetDouble = 4.8;
        bool foundDouble =
doubleVector.searchElement(searchTargetDouble);
        std::cout << "Search for element " <<
searchTargetDouble << ": " << (foundDouble ? "Found" :
"Not found") << std::endl;

        double averageDouble = doubleVector.findAverage();

```

```
        std::cout << "Average of elements in the vector: "
<< averageDouble << std::endl;
    } catch (const std::runtime_error& e) {
        std::cerr << "Error: " << e.what() << std::endl;
    }

    return 0;
}
```

OUTPUT

```
Smallest element in the vector: 1
Search for element 8: Found
Average of elements in the vector: 4.6
Smallest element in the vector: 1.2
Search for element 4.8: Found
Average of elements in the vector: 3.46
```

EXPERIMENT 9

```
#include <iostream>

template <typename T>
T findLargest(const T& a, const T& b, const T& c) {
    T largest = a;

    if (b > largest) {
        largest = b;
    }

    if (c > largest) {
        largest = c;
    }

    return largest;
}

int main() {
    // Example usage with integers
    int int1 = 10, int2 = 25, int3 = 15;
    int largestInt = findLargest(int1, int2, int3);
    std::cout << "Largest of " << int1 << ", " << int2 <<
    ", and " << int3 << ": " << largestInt << std::endl;

    // Example usage with doubles
    double double1 = 7.5, double2 = 12.8, double3 = 9.3;
    double largestDouble = findLargest(double1, double2,
double3);
    std::cout << "Largest of " << double1 << ", " <<
double2 << ", and " << double3 << ": " << largestDouble <<
std::endl;

    // Example usage with characters
    char char1 = 'A', char2 = 'Z', char3 = 'M';
    char largestChar = findLargest(char1, char2, char3);
    std::cout << "Largest of " << char1 << ", " << char2
<< ", and " << char3 << ": " << largestChar << std::endl;
}
```

```
    return 0;  
}
```

OUTPUT

Largest of 10, 25, and 15: 25

Largest of 7.5, 12.8, and 9.3: 12.8

Largest of A, Z, and M: Z